

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

INFORME INDIVIDUAL DE LAS EXPOSICIONES: HERRAMIENTAS CASE PARA GENERACIÓN DE CÓDIGO (MDD)

MATERIA:

INGENIERÍA EN REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

EQUIPO G

ESTUDIANTE:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, Correo: efuertes2@uteq.edu.ec

CURSO:

CUARTO SEMESTRE INGENIERÍA EN SOFTWARE “B”

Repositorio GitHub:

https://github.com/MiloSaurio4kHD/Informe_Exposiciones_Herramientas_Case_Fuertes_Grupo_G

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Introducción	3
2.	Grupo E – Modelo	3
2.1.	Descripción de la herramienta	3
2.2.	Demostración	3
2.3.	Observaciones	3
3.	Grupo H – BOUML	3
3.1.	Descripción de la herramienta	3
3.2.	Demostración	4
4.	Grupo D – Power Designer	4
4.1.	Descripción de la herramienta	4
4.2.	Demostración	4
5.	Cuadro comparativo de las herramientas	4
6.	Conclusiones	4

Índice de tablas

1.	Comparación de las herramientas CASE expuestas.	4
----	---	---

1 Introducción

El presente informe recoge, de forma individual, lo observado durante las exposiciones de los distintos equipos sobre herramientas CASE aplicadas al desarrollo dirigido por modelos (MDD), es decir, a la generación de código fuente a partir de diagramas UML. Cada equipo presentó una herramienta diferente y demostró, sobre el proyecto MundiPets, el ciclo de modelar un diagrama de clases y obtener a partir de él el esqueleto del código. El objetivo del documento es dejar constancia de las características técnicas de cada herramienta, del procedimiento demostrado en vivo y de las observaciones más relevantes sobre sus ventajas y limitaciones.

2 Grupo E – Modelio

Expositores: Nieves Jimmy y Morales Gary. **Sistema:** MundiPets.

2.1 Descripción de la herramienta

Nieves explicó que Modelio es una herramienta para desarrolladores y de código abierto. Morales complementó indicando que trabaja sobre UML, es compatible con UML2 y permite construir diagramas de clase y máquinas de estado, además de ser compatible con BPMN y ArchiMate. Señaló que Modelio depende de una clase para compilar sus códigos y que no es una inteligencia artificial, por lo que no corrige por sí sola los errores de UML; tampoco permite crear diagramas de perfil. Mencionó también que trae por defecto Java Classic y que la generación de código se realiza clase por clase.

2.2 Demostración

En la parte demostrativa, Nieves abrió Modelio y creó un proyecto. Para generar código a través de la clase se accede al apartado de módulo y, al crear un diagrama, se elige entre los distintos tipos disponibles el de clase. Allí aparecen diversos componentes: se arrastra el *class model* al área de trabajo y en ese mismo espacio se añaden atributos y métodos. Al dar clic sobre la clase se pueden modificar propiedades como el nombre de la clase y sus atributos; de la misma forma se agregan las operaciones y las relaciones entre clases.

A continuación, Morales mostró que para generar código se da clic derecho sobre una clase y se selecciona *Java element*, marcando así que la clase puede generar código Java, procedimiento que debe repetirse clase por clase. Con clic derecho sobre la clase aparece la opción *Java designer*, desde donde se realiza la generación. Luego se busca el directorio donde se guardó la clase, en la ruta

C:\usuarios\miusuario\modelio\nombreproyecto\src\, y se abre en el bloc de notas para ver el código. El mismo procedimiento se aplica al diagrama de clases del módulo de MundiPets.

2.3 Observaciones

Los expositores indicaron que el *roundtrip* no funciona de forma correcta porque el programa es muy antiguo, pero que las clases generadas se ejecutan sin problema en IntelliJ IDEA.

3 Grupo H – BOUML

Expositores: Chavarría, Mendoza y Sánchez Cornejo. **Sistema:** MundiPets.

3.1 Descripción de la herramienta

Chavarría explicó por qué el equipo eligió esta herramienta y qué iban a realizar, mencionando que utilizaron C++ para la generación de código. Mendoza describió el subsistema del diagrama UML de MundiPets, que contiene varias clases como Usuario, Propietario, Veterinario, Mascota e Historial Médico, con sus atributos y operaciones, además de la visibilidad y la multiplicidad. Señaló que BOUML permite ingeniería inversa, generación de código y trabajo multiplataforma, transformando diagramas UML a varios lenguajes como C++, C, Java, Python, PHP y SQL; añadió que tiene una interfaz sencilla y es de código abierto.

Sánchez matizó estas ventajas señalando limitaciones: la interfaz resulta poco intuitiva, la curva de aprendizaje es pronunciada y no posee documentación accesible. La generación de código es parcial, ya que solo genera las partes de la clase pero no las estructuras como tales; el trabajo con frameworks y bases de datos es manual. Advirtió además sobre una regeneración delicada, pues si se modifica y se vuelve a generar el código se elimina lo trabajado en esa clase. Concluyó que es gratuito pero cuenta con una comunidad desactualizada.

3.2 Demostración

Mendoza mostró que para generar un proyecto se abre BOUML y se pulsa *New*, se selecciona la ruta y se guarda, con lo que se crea la carpeta con todos los archivos del proyecto (lenguaje, estereotipos, etc.). Para crear un diagrama se da clic derecho al proyecto en BOUML y se crea el diagrama de clases; luego se selecciona la vista creada y se crea el diagrama, que se abre con doble clic. En la parte superior están todas las opciones: para crear una clase se selecciona *class* y se le asigna un nombre. Para crear un atributo se agrega y se le indica el nombre, y opcionalmente el tipo de dato y la privacidad. Para ver el tipo de dato y el valor que reporta, en *drawing settings*, dentro de *diagram*, se configura que no se oculten los atributos de las clases; de igual forma se puede agregar un estereotipo. Para relacionar clases, el menú superior contiene las relaciones de UML.

Para generar código se da clic derecho a la vista de clases y, en *edit*, se crea un *Source artefact*, procedimiento que se repite clase por clase, tras lo cual el código puede observarse en la carpeta. Finalmente se abre la terminal PowerShell, se navega a la carpeta y se ejecuta el *.exe* generado por BOUML.

Chavarría demostró la edición: para editar un atributo de una clase se da clic derecho sobre el atributo y se edita, o se agrega un nuevo atributo, y se añade un operador o método. Al guardar se observa que las cuatro clases fueron actualizadas y, en la carpeta generada por Mendoza, se ejecuta el comando para ver reflejado lo creado en las clases.

4 Grupo D – Power Designer

Expositor: Contreras Kevin. **Herramienta CASE:** Power Designer.

4.1 Descripción de la herramienta

Contreras Kevin explicó que Power Designer no se centra tanto en el modelado, sino en la conexión de los modelos.

4.2 Demostración

Primero creó el proyecto y explicó los diferentes diagramas que permite modelar Power Designer. Para la demostración seleccionó el diagrama de clases, en el cual creó las clases —como la clase Usuario con todas sus propiedades— y les agregó atributos y operaciones. Luego pasó a un diagrama que ya tenía modelado y lo exportó a código SQL, mostró el código generado en SQL Server y lo ejecutó para que se crearan las tablas.

5 Cuadro comparativo de las herramientas

La Tabla 1 resume las herramientas presentadas por los distintos equipos, según lo observado en cada exposición.

Tabla 1: Comparación de las herramientas CASE expuestas.

Herramienta	Lenguaje demostrado	Fortalezas observadas	Limitaciones observadas
Modelio	Java	Código abierto; compatible con UML2, BPMN y ArchiMate; clases ejecutables en IntelliJ IDEA.	Generación clase por clase; <i>roundtrip</i> no funciona bien por ser un programa antiguo; no crea diagramas de perfil.
BOUML	C++	Código abierto y gratuito; multiplataforma; genera a C++, C, Java, Python, PHP y SQL; permite ingeniería inversa.	Interfaz poco intuitiva; curva de aprendizaje pronunciada; generación parcial; regeneración delicada (borra lo trabajado); comunidad desactualizada.
Power Designer	SQL	Orientada a la conexión de modelos; exporta a SQL y ejecuta la creación de tablas en SQL Server.	Menor énfasis en el modelado según lo expuesto.

6 Conclusiones

Las exposiciones evidenciaron que, si bien todas las herramientas comparten el propósito de generar código a partir de modelos UML, difieren notablemente en cobertura de lenguajes, facilidad de uso y calidad del ciclo de sincronización modelo-código. Modelio y BOUML coinciden en ser opciones de código abierto orientadas a UML, aunque ambas realizan la generación clase por clase y presentan limitaciones en el *roundtrip* y la regeneración. Power Designer se distingue por su enfoque hacia la conexión de modelos y la generación directa

de esquemas SQL ejecutables. En conjunto, las demostraciones permitieron comparar distintos enfoques de MDD sobre un mismo caso —MundiPets— y reconocer que la elección de la herramienta depende del lenguaje objetivo, del alcance del proyecto y del nivel de personalización requerido en la salida generada.